CIS-11 Project Documentation

Calculator Hater

Eric Espinoza & Manuel Nunez

Course Project: Case B (Test Score Calculator)

May 24, 2026

Advisor: Kasey Nguyen, PhD

Part I - Application Overview

Objectives

This project is to create a program that displays the minimum, maximum, and average grade of 5 test scores and display the letter grade associated with the test scores.

Any user will be able to calculate test score details and be shown corresponding letter grade values very quickly and effortlessly.

The program is relatively simple to create and will yield valuable time-efficiency benefits.

Business Process

This program, used in a business context, will reduce time and effort spent calculating test score details and result in employees spending time working on higher-value tasks.

User Roles and Responsibilities

Role	Objective	Tasks	Relationship
Class Instructor	Maintain accurate records and evaluate performance.	1. Input test scores into system 2. Prompt calculations for min/max/average 3. Verify letter grades	Hands off test score details for distribution to students and the Registrar.
System Administrator	Ensure accurate processing for student administration and academic integrity.	1. Set grade thresholds 2. Verify inputs 3. Maintain records	Receives reports and verifies that logic and calculations are within policy standards.

Production Rollout Considerations

Rollout Strategy: Phased deployment starting with pilot testing to verify accuracy.

Data Population: Initial data populated via manual entry.

Data/Transaction Volume: Capable of 5 scores/session. Expected use is per test.

Terminology

Test score: a value between 0 and 100 representing a student's performance on a single test.

Average: The mean of the input set; the sum of all scores divided by the total number of scores.

Letter grade: A qualitative label (A, B, C, D, or F) assigned to test scores based on position within a set numerical range.

Part II - Functional Requirements

Statement of Functionality

User Input:

- The app will ask the user to enter exactly 5 test scores one by one.
- It will only accept numbers (`0` to `9`) typed from the keyboard.
- Pressing the **Enter** key will finish the input for each score.
- The app will display each number on the screen as the user types it.

Data Storage:

- The app will reserve a 5-slot memory array to save the scores.
- It will use a memory pointer to automatically find the next empty slot for each score.

Calculations:

- The app will find the lowest score (Minimum).
- The app will find the highest score (Maximum).
- It will add all 5 scores together to get a total sum.
- It will divide the total by 5 to find the average. Any decimal remaining will be dropped (rounded down to a whole number).

Letter Grade Rules:

- The final letter grade will be based entirely on the calculated average.
- The grade scale is strictly defined as:
    - **A:** 90 to 100
    - **B:** 80 to 89

    - **C:** 70 to 79
    - **D:** 60 to 69
    - **F:** 0 to 59

Output Display:

- The screen will display the final Minimum, Maximum, Average, and Letter Grade.
- Each result will have a clear text label (e.g., `"Minimum Score: "`).
- The internal binary data will be converted to readable text before printing.
- Each result will print on a fresh, separate line.
- The program will stop completely (`HALT`) after printing the final grade.

Coding & System Constraints

- **Subroutines:** The code must be modular. Input conversion, division, output printing, and grade mapping must live in their own separate, reusable subroutines.
- **Stack Protection:** Subroutines must use a memory stack (`PUSH` and `POP`) to save and restore register data so they don't overwrite or corrupt active values.
- **Input Range:** The program assumes all entered scores are already valid integers between 0 and 100. Error checking for invalid inputs is excluded from this version.

Scope

The LC-3 Grade Processor will be developed and delivered in a single, complete deployment phase.

- **In Scope:**
    - Interactive console prompts for exactly 5 test scores.
    - Automatic identification of the highest and lowest scores/
    - Calculation of the integer average using an iterative subtraction division subroutine.
    - Display of a single letter grade (A,B,C,D, or F) based on the average.
    - Proper memory stack management (PUSH/POP) to preserve register data.

- **Out of Scope:**
    - Processing more or fewer than 5 test scores.
    - Input validation for negative numbers, non-numeric characters, or scores greater than 100.
    - Handling or displaying fractional decimal points for the average score.
    - Storing or saving grade history across multiple application runs.

Performance Requirements:

- **Calculation Speed:** The application shall compute the minimum, maximum, and average scores instantly (in less than 1 second) after the user inputs the 5th score.

- **Output Display Speed:** The final results and letter grade shall print to the console with zero visible delay or text lagging.

Usability Requirements

- **User Prompts:** The system shall display a clear instruction before every input so the user knows exactly what to type (e.g., "Enter test score (0-100) and press Enter: ").
- **Visual Feedback:** The console shall immediately echo back characters as the user types them so they can see what they are entering.
- **Clean Layout:** The system shall print each final metric (Min, Max, Average, and Letter Grade) on its own separate line using standard labels so the results are easy to read.

Part III - Appendices

Pseudo-code

```
BEGIN MAIN
    INITIALIZE stack pointer (R6)

    ; Setup for input
    CALL InputScoresSubroutine

    ; Logic
    CALL CalculateStatsSubroutine

    ; Output
    CALL DisplayResultsSubroutine

    HALT
END MAIN

BEGIN InputScoresSubroutine
    FOR i = 0 TO 4:
        PRINT "Enter score: "
        GETC (system call to get input)
        CONVERT ASCII to integer
        STORE in Array[i]
        PUSH score onto Stack
    ENDFOR
```

```
    RET
END InputScoresSubroutine

BEGIN CalculateStatsSubroutine
    ; Save-Restore (Regs R1-R3)
    PUSH R1, R2, R3

    SET Min = 100, Max = 0, Sum = 0
    FOR i = 0 TO 4:
        Load score from Array[i]
        Sum = Sum + score
        IF score > Max THEN Max = score
        IF score < Min THEN Min = score
    ENDFOR

    Average = Sum / 5

    POP R3, R2, R1
    RET
END CalculateStatsSubroutine

BEGIN DisplayResultsSubroutine
    PRINT "Max: ", Max
    CALL ConvertAndPrintLetter(Max)

    PRINT "Min: ", Min
    CALL ConvertAndPrintLetter(Min)

    PRINT "Average: ", Average
    CALL ConvertAndPrintLetter(Average)
    RET
END DisplayResultsSubroutine

BEGIN ConvertAndPrintLetter(Value)
    IF Value >= 90 THEN PRINT "A"
    ELSE IF Value >= 80 THEN PRINT "B"
    ELSE IF Value >= 70 THEN PRINT "C"
    ELSE IF Value >= 60 THEN PRINT "D"
    ELSE PRINT "F"
    RET
```

```
END ConvertAndPrintLetter
```